

Documento de Requisitos

Información General

Nombre del Proyecto: GestiónEmpleados

Equipo de Desarrollo:

Javier Galante Gómez

Descripción del Proyecto

El proyecto consiste en el desarrollo de una aplicación de consola robusta y modular en Java orientada a objetos, diseñada para automatizar y administrar las operaciones de recursos humanos y el control financiero de una organización. El software sigue una arquitectura limpia inspirada en el patrón Modelo-Vista-Controlador (MVC), lo que garantiza una separación estricta entre la lógica de los datos, el control de las reglas de negocio y la interfaz de usuario.

El núcleo del sistema reside en el paquete "modelo", estructurado en torno a una jerarquía de herencia que resuelve el problema de la diversidad de contrataciones. La pieza central es la clase abstracta Empleado, la cual define las propiedades transversales de cualquier trabajador (DNI, nombre, departamento, fecha de alta, credenciales, etc.) y establece el método abstracto calcularSalarioBruto(). De esta superclase se derivan tres subclases especializadas que encapsulan las lógicas particulares de cada nómina mediante polimorfismo dinámico:

- EmpleadoAsalariado: Gestiona trabajadores con sueldo fijo y complementos.
- EmpleadoPorHoras: Calcula la retribución multiplicando las horas trabajadas por el precio por hora estipulado.
- EmpleadoComisionista: Determina el salario combinando una base mínima garantizada con un porcentaje variable sobre las ventas realizadas en el mes.

Se hace uso de la interfaz Evaluable, implementada por la clase base Empleado. Esta interfaz estandariza el registro de puntuaciones de rendimiento y la clasificación del estado de desempeño ("Excelente", "Satisfactorio", "Requiere Mejora"), integrando la gestión del talento en el flujo del programa.

La organización de los trabajadores y los departamentos está centralizada en la clase Empresa, que actúa como contenedor de datos mediante colecciones dinámicas (List y HashMap). Por su parte, la creación y validación de las distintas instancias de trabajadores se delega en el componente EmpleadoBuilder. El patrón de diseño (Builder) asegura que no se inserten objetos corruptos o con datos negativos en el sistema, centralizando reglas estrictas de validación de negocio antes de la instanciación (Para la carga inicial de datos no se ha utilizado el builder ya que se supone que los datos a cargar son correctos).

El paquete “controlador” (a través de sus componentes: Autenticador y ControladorEmpresa) gestiona la seguridad y los casos de uso principales. El sistema implementa un control de acceso por perfiles que divide las funcionalidades en dos interfaces controladas por VistaConsola:

1. Perfil Administrador: Permite realizar tareas de alta gestión, tales como la búsqueda, adición, eliminación o modificación de empleados así como listar los departamentos o calcular los costes financieros globales. Destaca la función calcularCosteTotalEmpresa(), la cual computa de manera automatizada las bases de cotización de la seguridad social y los tipos de accidentes según el riesgo asociado a cada departamento (por ejemplo, aplicando tasas más altas a personal de Sistemas o Ventas).
2. Perfil Empleado: Ofrece un espacio personal restringido mediante contraseña (personal de cada trabajador) donde el usuario puede visualizar el desglose de su nómina, consultar su horario laboral, realizar el fichaje de entrada o salida del día (que está conectado con su evaluación) o introducir las horas trabajadas y ventas realizadas según el tipo de empleado.

La capa “vista” gestiona toda la interacción por consola de forma segura mediante técnicas de captura y limpieza de búfer, mitigando excepciones por datos inválidos introducidos por teclado, como indica el patrón de diseño MVC.

Requisitos Funcionales

Actores del Sistema

Administradores y Empleados

Casos de Uso o Funcionalidades

Administrador:

1. Autenticar Administrador: El componente Autenticador valida la autenticación del administrador que se muestra como una pantalla de “login”.

2. Mostrar listado de empleados: Muestra un listado de todos los empleados de la empresa.
3. Mostrar listado ordenado de empleados (antigüedad y desempeño): Muestra un listado de todos los empleados de la empresa ordenados según un criterio.
4. Búsqueda de Empleado (único): Las búsquedas por ID y DNI devuelven un único empleado y lo muestran por pantalla.
5. Búsqueda de empleados (listado): Las búsquedas por Nombre, Apellido y Email devuelven un listado de empleados que contienen el parámetro buscado en el campo correspondiente.
6. Mostrar todos los empleados de un Departamento: La búsqueda por departamento (por la abreviatura del mismo) devuelve el listado de empleados pertenecientes a ese departamento y los muestra por pantalla.
7. Calcular gasto de empresa en un empleado (al mes): Busca un empleado por ID y calcula el gasto que supone a la empresa (salario bruto + impuestos a cargo de la empresa).
8. Calcular gasto de empresa en un departamento (al mes): Calcula el gasto del departamento sumando el gasto de todos los empleados pertenecientes al mismo (salario bruto + impuestos a cargo de la empresa).
9. Calcular gasto total de la empresa (al mes): Calcula el gasto total sumando el gasto de todos los empleados (salario bruto + impuestos a cargo de la empresa).
10. Añadir nuevo empleado "asalariado": Pide datos por pantalla, genera un nuevo empleado del tipo deseado y lo añade al listado de empleados de la empresa.
11. Añadir nuevo empleado "por horas": Pide datos por pantalla, genera un nuevo empleado del tipo deseado y lo añade al listado de empleados de la empresa.
12. Añadir nuevo empleado "comisionista": Pide datos por pantalla, genera un nuevo empleado del tipo deseado y lo añade al listado de empleados de la empresa.
13. Eliminar empleado: Busca por ID y elimina el empleado de la lista de empleados de la empresa.

14. Modificar empleado: Busca por ID, pide los datos a modificar proporcionando sus parámetros para mayor comodidad y los modifica.
15. Mostrar listado de departamentos: Muestra un listado de todos los departamentos de la empresa y el número de empleados que tiene.

Empleado:

16. Autenticar Empleado: El componente Autenticador valida la autenticación del empleado que se muestra como una pantalla de "login" en la que poner el ID del empleado y su contraseña (cada empleado tiene su propia contraseña).
17. Consultar nómina: Consulta la nómina (salario bruto) del empleado desglosado según el tipo de empleado.
18. Consultar Horario: Consulta el horario (hora de entrada y hora de salida) del empleado. Los empleados "por horas" no tienen horario estipulado.
19. Cambiar contraseña: Pregunta por la contraseña actual y permite cambiarla a otra diferente.
20. Registro de entrada/salida: Permite al empleado "asalariado" y al empleado "comisionista" fichar a la hora que entran y salen de la oficina.
21. Registro de horas trabajadas: Permite al empleado "por horas" que indique las horas que ha trabajado en el día y que se le sumarán a su acumulado de horas del mes.
22. Registro de ventas realizadas: Permite al empleado "comisionista" que indique las ventas que ha realizado en el día y que se le sumarán a su acumulado de ventas del mes.

Sistema:

23. Precarga de datos: precarga de una serie de departamentos y empleados en el sistema para que podamos trabajar con ellos (se ha optado por una precarga desde el mismo código y no desde archivos o bases de datos externas).

Requisitos No Funcionales

1. Arquitectura basada en Patrones (MVC): Sistema separado en componentes (lógica de negocio, modelo de datos e interfaz de usuario).
2. Encapsulamiento de los datos: Se ha intentado encapsular atributos y métodos en la medida de lo posible para asegurar su accesibilidad por

parte de otras partes del código. Además de utilizar getters y setters cuando es necesario.

3. Autenticación y seguridad por perfiles: Distinguimos el acceso de administrador por el acceso de empleado cada uno protegidos por usuario y contraseña.
4. Utilización del Patrón Builder: A la hora de añadir un nuevo empleado se hace uso de la clase EmpleadoBuilder que simplifica la creación de los distintos tipos de empleado.
5. Ocultación de credenciales en interfaz: Las contraseñas de los usuarios no se exponen de forma abierta en los volcados de texto generales de la plantilla.
6. Tolerancia a Fallos en la Entrada de Datos (Validación de Tipos): Validación y control en las entradas de datos que hace el usuario por teclado mediante funciones auxiliares en la vista.
7. Integridad de Datos en la Instanciación: A la hora de crear o modificar empleados, el sistema comprueba que la información es correcta para su inserción o guardado para evitar fallos.
8. Gestión de Memoria Dinámica: Se han utilizado Colecciones dinámicas propias de Java en lugar de arrays estáticos de tamaño fijo para el guardado y manipulación de la información.
9. Portabilidad (Independencia de Plataforma): La aplicación es capaz de ejecutarse y funcionar correctamente en cualquier sistema operativo.
10. Interfaz de Usuario Autocontenida (Usabilidad por Consola): La interfaz de texto guía correctamente y de manera sencilla al usuario a través de los menús y opciones además de confirmar cada acción para un mejor entendimiento del funcionamiento.
11. Rendimiento acorde: El rendimiento de la aplicación es correcto y consistente para todas las operaciones.
12. Escalabilidad: La aplicación está diseñada para que pueda expandirse añadiendo funcionalidades, cambiando la vista por otra diferente, cambiando la precarga de datos...